59

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Çoklu Ortam Dağıtımı (Dev -> Staging -> Prod) ve Canary

Multi-Stage Environment Promotions: Dev, Staging & Production

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Yeni geliştirilen bir ilacı önce laboratuvarda fare üzerinde (Dev ortamı), sonra gönüllü küçük bir grupta (Staging ortamı), her şey yolundaysa tüm dünyada (Production ortamı) kullanıma sunarsınız! Hatta üretimde bile önce müşterilerin sadece %10'una yeni versiyonu gösteren 'Kanarya' (Canary) testi yaparsınız. Bir hata varsa sadece %10 etkilenir, anında geri çekilir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't test a brand-new experimental medicine on the entire global population on day one! You test in the lab (Dev), validate with a small volunteer trial (Staging), and only then release to the public (Production)! And even in production, you use a 'Canary' release: route 10% of users to the new version first. If error rates tick up, kill it instantly before 90% notice!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kustomize 'Overlays' mimarisi ile ortam yönetimi: `base/` (Ortak Deployment ve Service tanımları) üzerine `overlays/dev/` (1 kopya, düşük CPU), `overlays/staging/` ve `overlays/production/` (10 kopya, yüksek CPU, HPA, katı kısıtlar) bindirilir. GitHub Actions 'Environments' mekanizmasıyla Production ortamına geçiş için kıdemli mühendis onayı (Manual Approval Gate) zorunlu kılınabilir. Flagger ile Canary analizleri (HTTP hata oranı <%1) yürütülür.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Multi-tier promotion leverages Kustomize Overlays: a common `base/` manifest stack is patched by `overlays/dev/`, `overlays/staging/`, and `overlays/production/` (varying replicas, resource limits, and secrets). GitHub Environments enforce protection rules requiring manual engineering lead sign-offs before staging can promote to production.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Kustomize Ortam Ağacı:
# apps/coffee-app/
# ├── base/
# │   ├── deployment.yaml
# │   └── service.yaml
# └── overlays/
#     ├── dev/
#     │   └── kustomization.yaml (replicas: 1)
#     └── production/
#         └── kustomization.yaml (replicas: 5, HPA aktif)

# Production Terfisi İçin GitHub Actions Onay Adımı:
# environment:
#   name: production
#   url: https://coffee.production.company.com
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Farklı ortamlar için ayrı ayrı devasa YAML kopyaları oluşturmayın (Kopya-Yapıştır kirliliği!). Ortak şablonu `base/` altında tutun ve sadece değişen ayarları `overlays/` içinde yamalayın (DRY - Don't Repeat Yourself).

🇬🇧 WATCH OUT

Never duplicate massive YAML manifests across environments! Maintain shared definitions in `base/` and patch environment deltas inside `overlays/`.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Kustomize overlays tek bir temel şablonla farklı ortamları yönetir.
- GitHub Actions Onay Kapıları (Approval Gates) kontrolsüz prod geçişlerini durdurur.
- Kanarya (Canary) dağıtımları hata anında felaketi önceden yakalar.

- Kustomize overlays eliminate manifest duplication while supporting environment nuances.
- Manual approval gates prevent unverified releases from hitting production.
- Canary deployments limit blast radius to a microscopic fraction of live traffic.